



Nirish Samant

Dynamic and Adaptive Attention in Modern Transformers

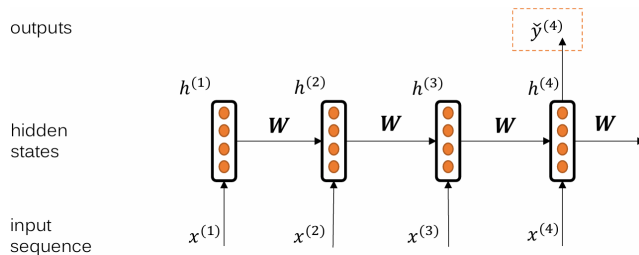
Supervisor: Lauritz Rasbach

Table of contents

- 1 Motivation
- 2 Transformer Attention
- 3 Transformers Modern Architectures
- 4 Attention Interpretability
- 5 Conclusion

Recurrent Neural Networks (RNNs)

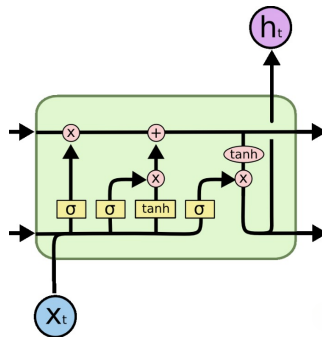
- RNNs suffer from **sequential** architecture issues.
- Sequential computation **inhibits parallelization**.
- The longer the sequence is, the more probable the information is to be lost along the chain (**vanishing gradients**).
- More influence of the **last states**.
- Difficult to train for **large amounts of text**.



Source: Giplab, "Deep Learning for Natural Language Processing"

Long Short Term Memory (LSTM)

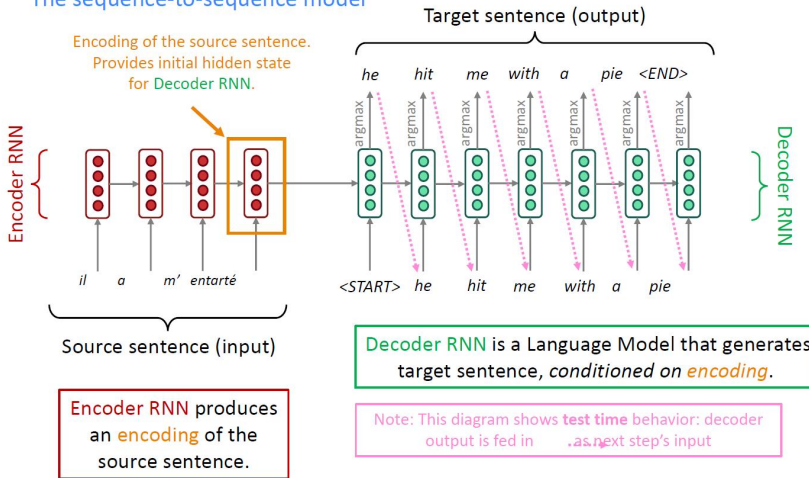
- LSTMs mitigate some RNN issues.
- Allows us to deal with the Vanishing Gradient problem.
- However, LSTMs do not scale. The **“memory” issue** is still present.
- Sequential computation **inhibits parallelization**.



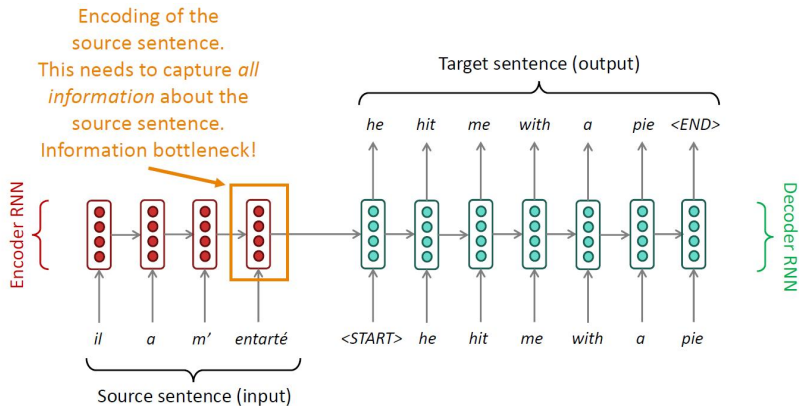
Source: Christopher Olah, "Understanding LSTM Networks" (2015)

Seq2Seq Model

The sequence-to-sequence model



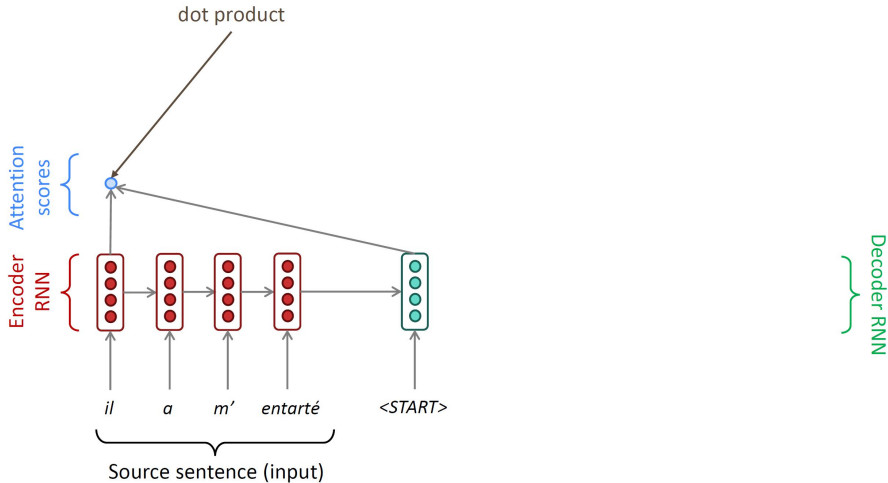
Issue in Seq2Seq



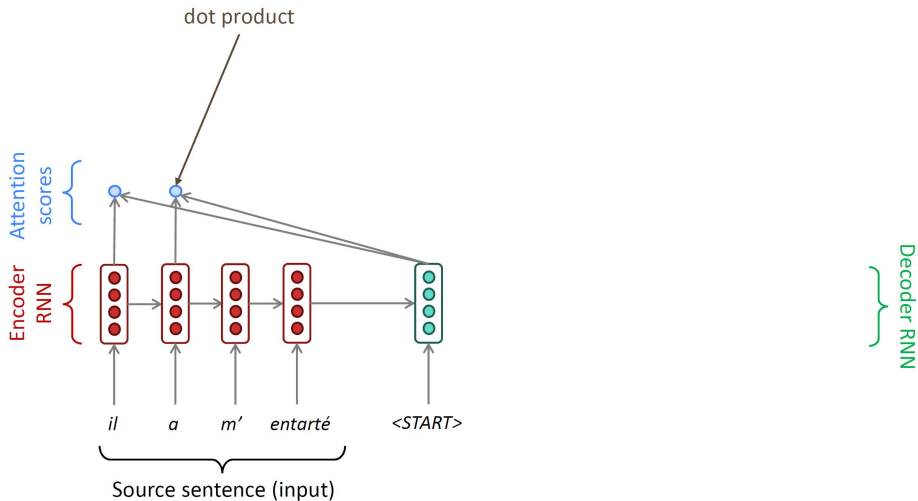
Attention

- Attention allows the decoder to **look directly at the source; bypass bottleneck.**
- Attention helps with the **vanishing gradient problem**. Provides a shortcut to faraway states.
- Selectively focus on **relevant information**.
- Prioritize **input features** depending on the **output to generate**.
- Allow the model to **ignore irrelevant information**.
- Attention provides some **interpretability**.

Attention

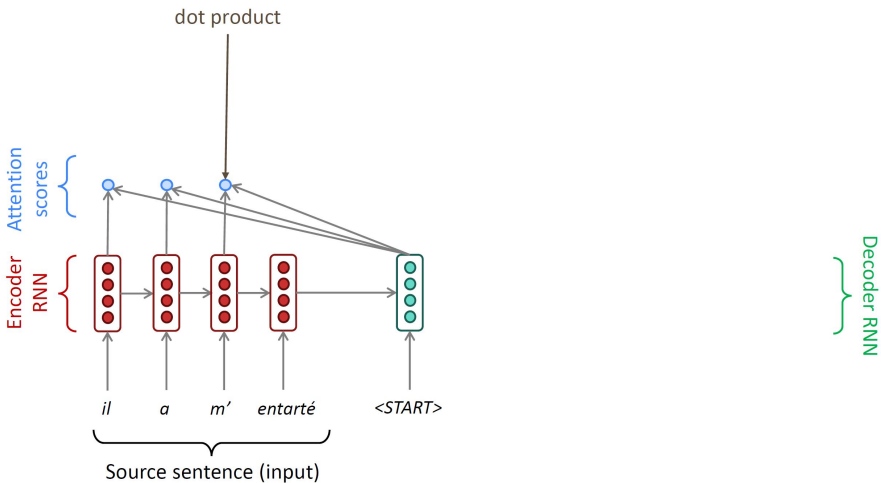


Attention

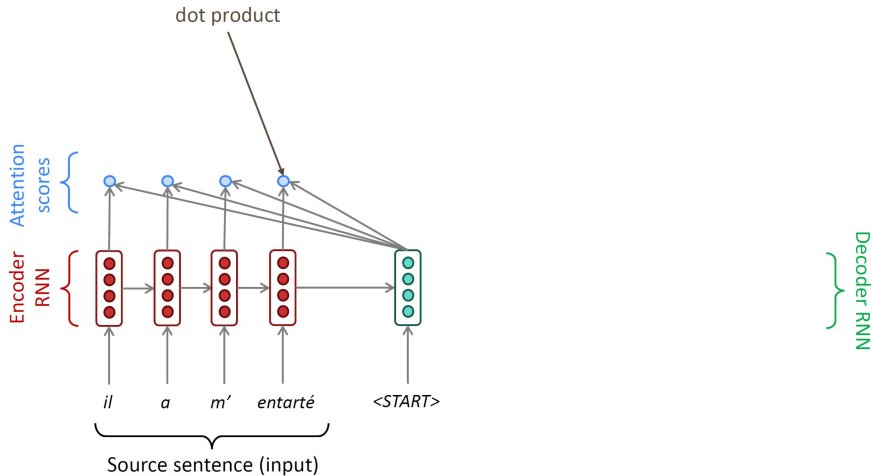


Source: University of Washington, CSE 543 (2023)

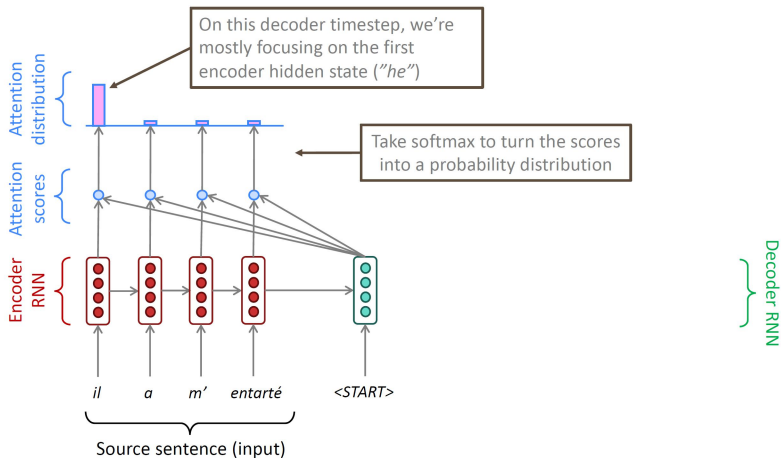
Attention



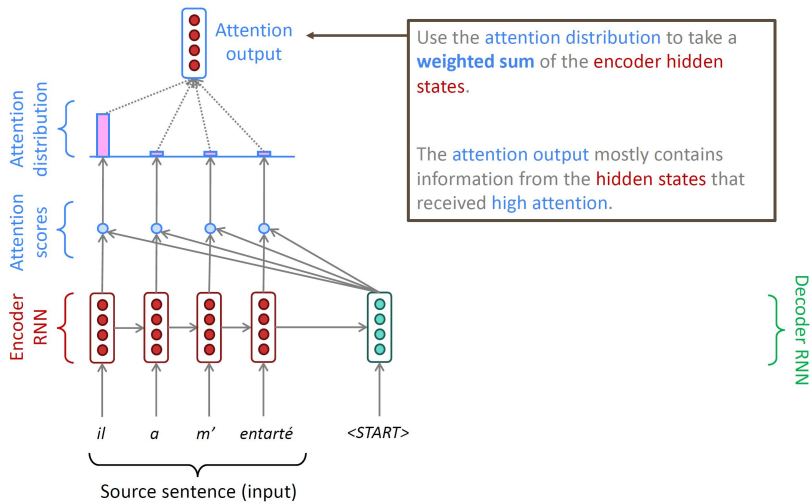
Attention



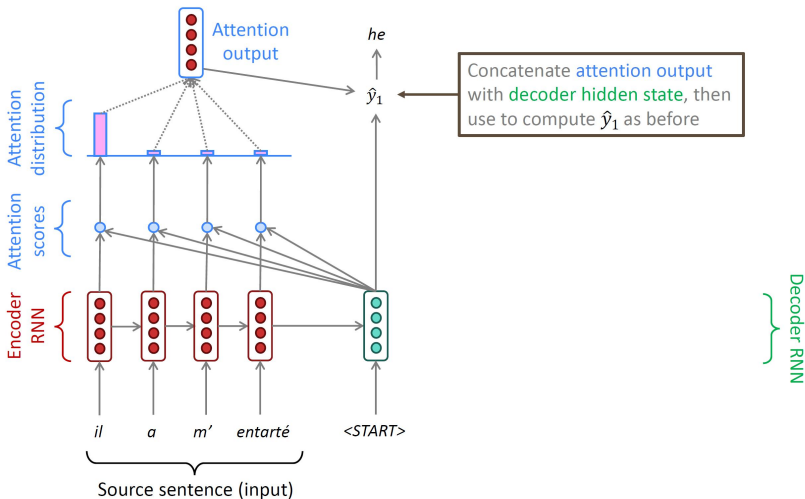
Attention



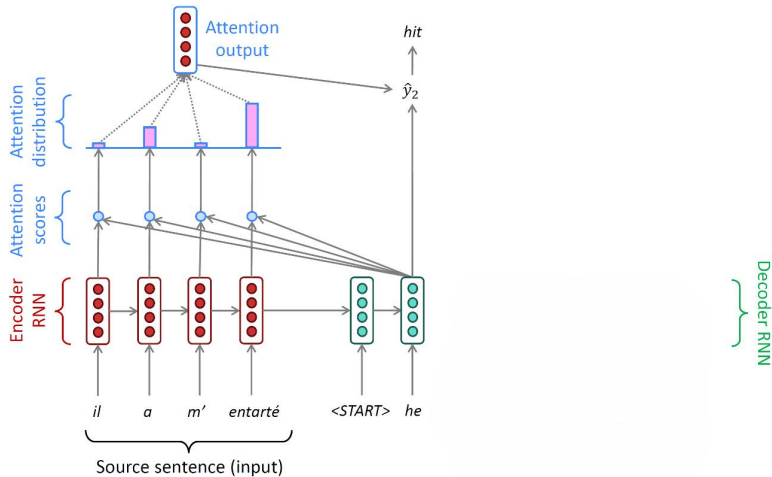
Attention



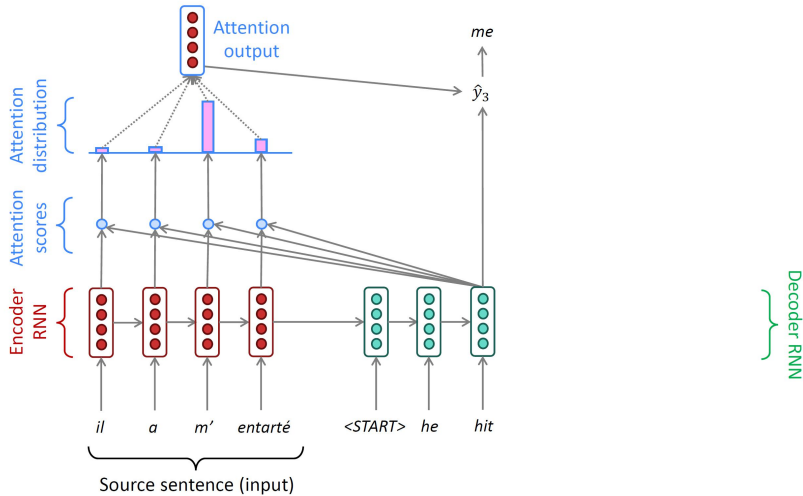
Attention



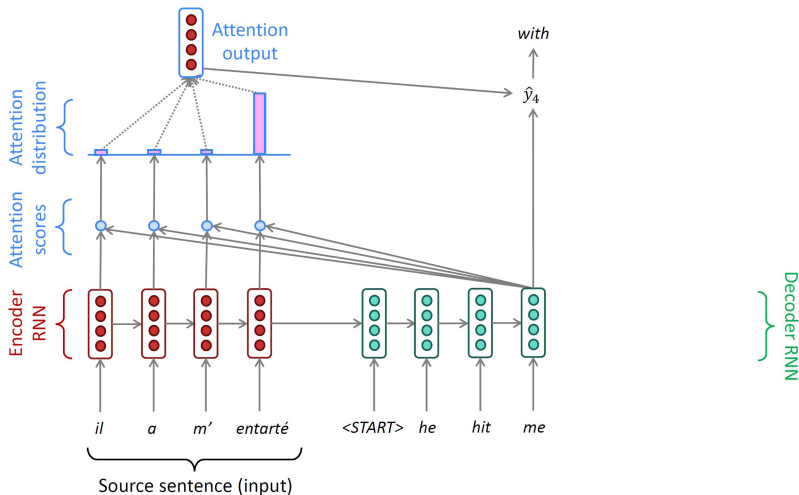
Attention



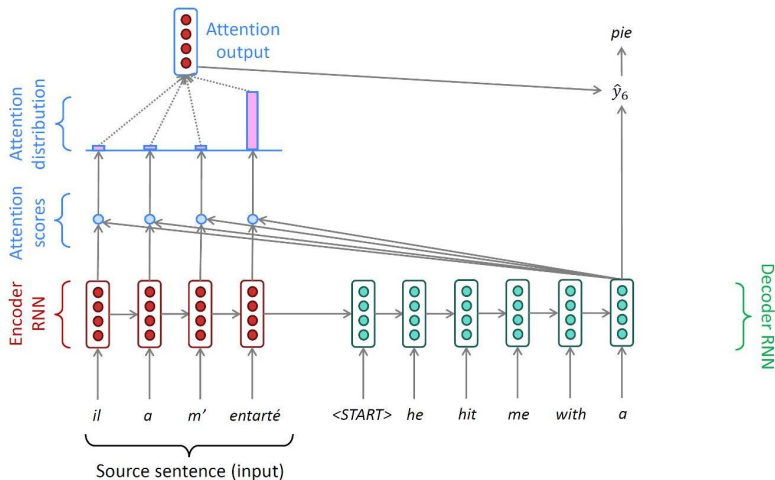
Attention



Attention



Attention



Attention

- We have **encoder hidden states**

$$h_0, \dots, h_N \in \mathbb{R}^h$$

- We have **decoder hidden state**

$$s \in \mathbb{R}^h$$

- We get the **attention scores** (energy or alignment scores) for the step T

$$e = [s^T h_0, \dots, s^T h_N] \in \mathbb{R}^N$$

Attention

- Take the **softmax** to get the **attention distribution** α

$$\alpha = \text{softmax}(e) \in \mathbb{R}^N$$

- We use **attention distribution** α to take a weighted sum of the encoder hidden states to get the **attention output** a

$$a = \sum_{i=1}^N \alpha_i h_i \in \mathbb{R}^N$$

- The **attention output** a is sometimes also called the **context vector**.

Transformers

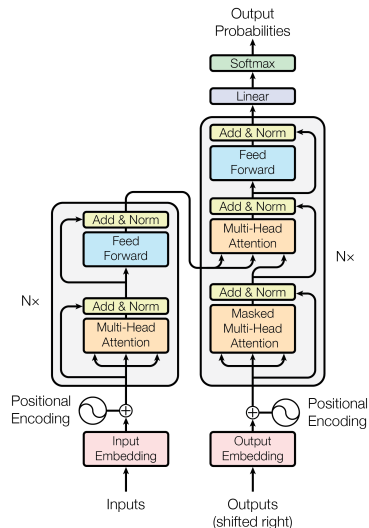
- The **Transformer** is a neural network architecture based entirely on **attention mechanisms** instead of recurrence or convolution.

- Consists of:

- ▶ Encoder (context understanding)
- ▶ Decoder (autoregressive generation)

- Core building blocks:

- ▶ Multi-Head Self-Attention
- ▶ Feed-Forward Networks
- ▶ Residual Connections + Layer Normalization

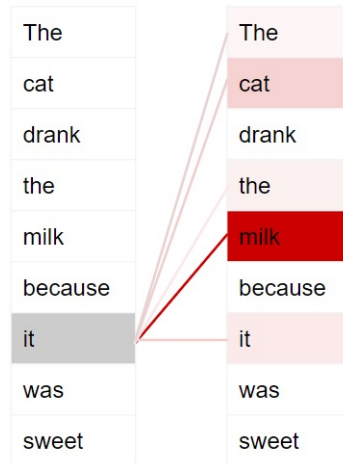
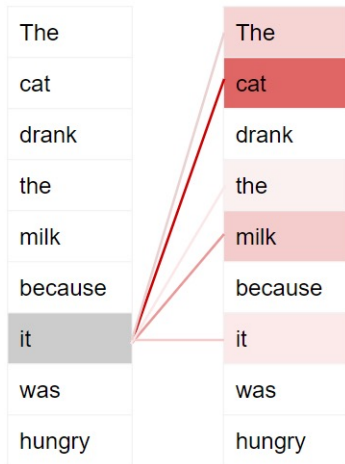


Source: Vaswani et al., "Attention Is All You Need" (2017)

Self-Attention

- **Self-Attention** is a mechanism that allows each token in a sequence to **attend to all other tokens**.
- Self-Attention prioritizes certain text parts based on their **context**.
- It computes **relevance scores** between tokens using similarity in a shared embedding space.
- It computes dynamic importance weights based on **similarity** between token representations.
- Enables modeling of **long-range dependencies** without recurrence.

Self-Attention



Self-Attention Weight Matrices & Matrix Operations

Each input x_i is multiplied with three different matrices:

query

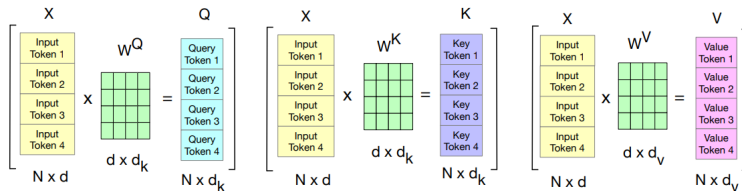
$$q_i = x_i W^Q$$

key

$$k_i = x_i W^K$$

value

$$v_i = x_i W^V$$



- W^Q and W^K both have $d \times d_k$ dimensions, W^V has $d \times d_v$ dimensions.
 - ▶ d , d_k , and d_v are **hyperparameters**
 - ▶ d_k and d_v are typically much smaller than the input dimension d
 - ▶ **Learnable:** The weight matrices are **optimized during training**.

Source: Prof. Lisa Beinborn, "Interpretability and Bias of Machine Learning Models"

Self-Attention

■ Create **three matrices**

- ▶ Query
- ▶ Key
- ▶ Value

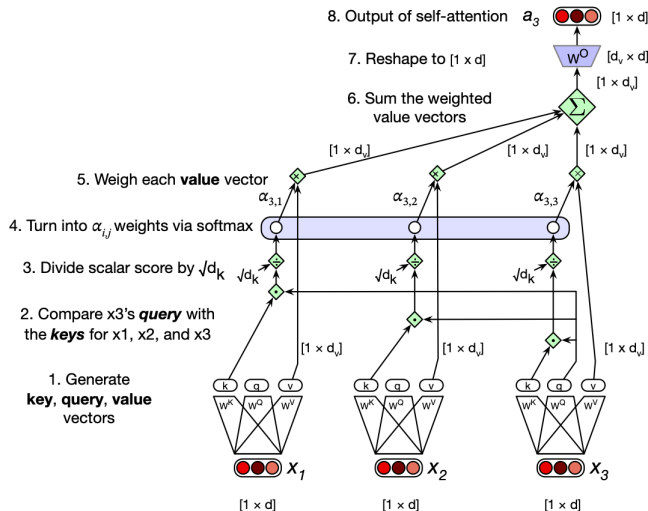
■ Initially, weights for Q, K, and V are **randomly initialized** and **learned during training**.

■ Output is a **weighted sum**, where...

- ▶ ...weight of each value is computed by an inner product of Q and K.
- ▶ ...queries and keys have same dimensionality d_k and values have d_v .

$$A(Q, K, V) = \mathbf{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Self-Attention Overview



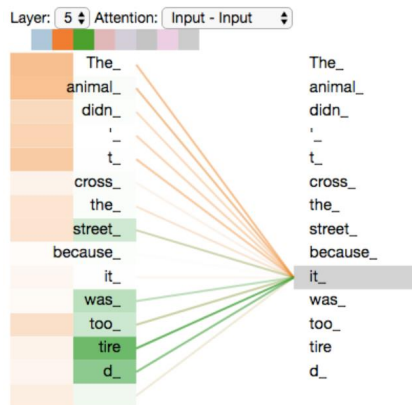
Source: Jurafsky & Martin, "Speech and Language Processing"

Multi-Head Attention

- Perform attention **multiple** times.
- If the network learns multiple attention heads (= multiple sets of Q, K, V matrices), it might learn to attend to **different aspects of language**, like local semantics, grammatical gender agreement, coreference resolution, number agreement, semantic roles,
- For example, **BERTbase calculates 12 attention heads** in each self-attention layer. The 12 attention matrices are concatenated and multiplied by a weight matrix.
- Each head calculates a separate set of attention vectors that are then **combined**.

Multi-Head Attention

- Here, different colors show the attention of different heads.
- Figure shows: Attention heads 2 (orange) and 3 (green) in layer 5.



Source: Jay Alammar, "The Illustrated Transformer" (2018)

Multi-Head Attention

- Perform attention multiple times (e.g., eight)
 - ▶ Produce eight vectors out of the input word vectors and Q, K, and V
 - ▶ **Split the embedding space** between the attention heads

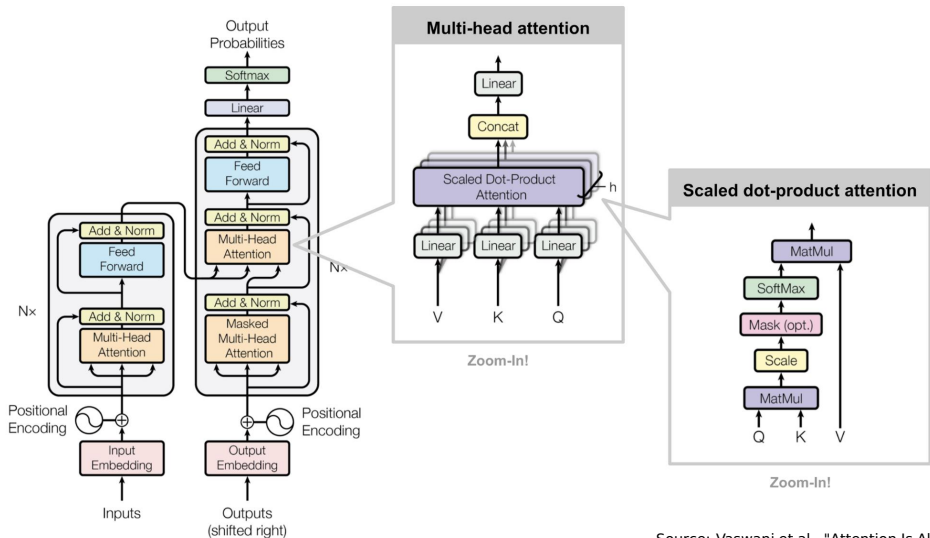
$$d_k = \frac{d_{\text{model}}}{h}$$

- To condense these matrices (e.g., eight) into a single one, they concatenate all $head_i$ and multiply by a weight matrix W^O

$$\text{MultiHead}(Q, K, V) = \text{Concat}(head_1, head_2, \dots, head_n)W^O$$

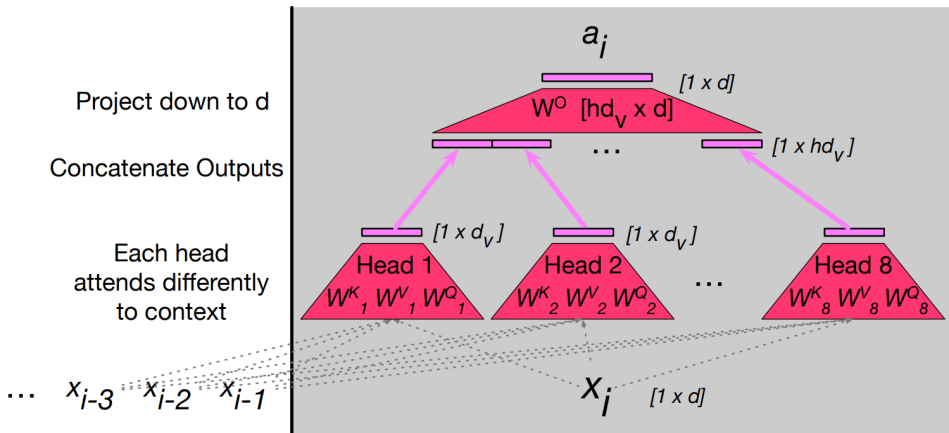
$$head_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Multi-Head Attention



Source: Vaswani et al., "Attention Is All You Need" (2017)

Multi-Head Attention



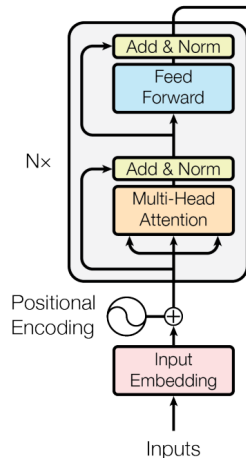
Attention in Transformer Encoder

- The encoder uses **Multi-Headed self-attention**.
- Queries, Keys, and Values are all computed from the **encoder input embeddings**:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

- The **attention weights** determine how much information each token gathers from every other token:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

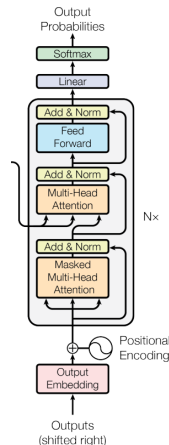


Attention in Transformer Decoder

- The decoder first applies **masked multi-head self-attention** to previously generated tokens.
- A **causal mask** prevents a token from attending to future positions, ensuring **autoregressive generation**.
- The decoder then performs **encoder-decoder (cross) attention**, where:

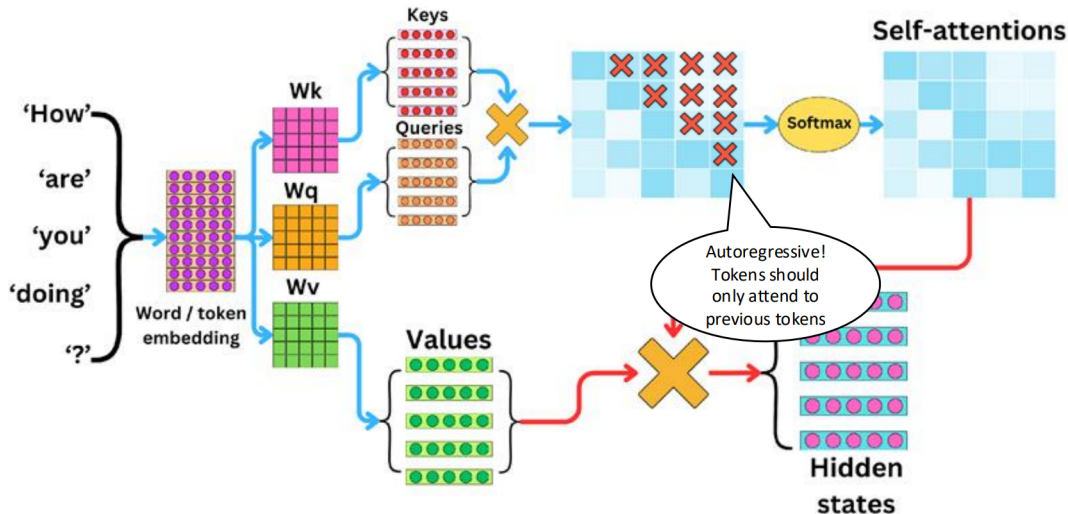
$$Q = \text{Decoder Hidden States}, \quad K, V = \text{Encoder Hidden States}$$

- This allows each generated token to attend to the relevant parts of the input sequence while producing the next output token.

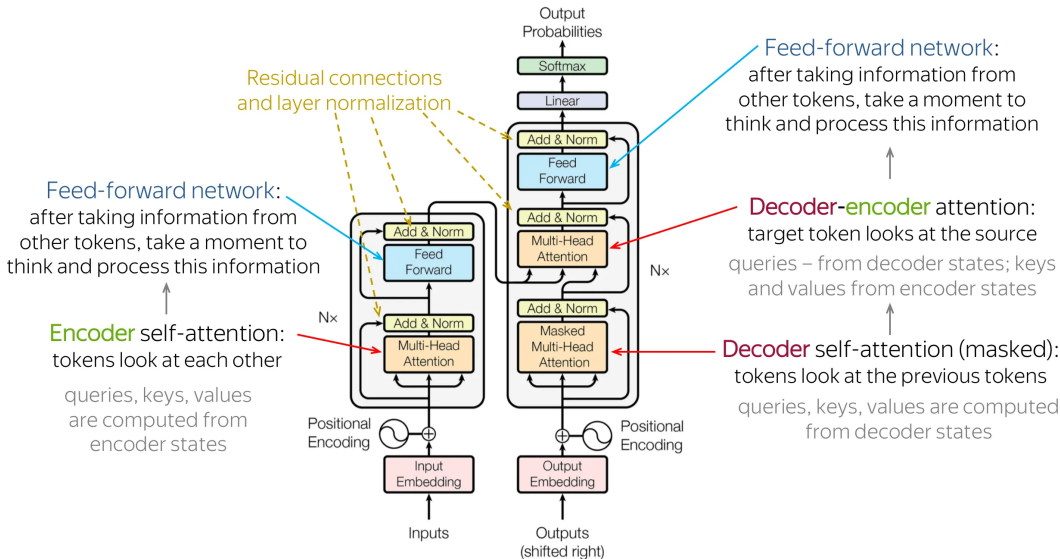


Source: Vaswani et al., "Attention Is All You Need" (2017)

Attention in Transformer Decoder (Masking)



Source: <https://newsletter.theaiedge.io/p/understanding-the-self-attention>



Source: Primers Transformers, Aman AI, <https://aman.ai/primers/ai/transformers/>

RoPE (Rotary Positional Embeddings)

- Transformers need **positional information** because attention itself is **permutation-invariant**.
- RoPE encodes position using **rotations in vector space**.
- The key idea: position is injected by rotating **query and key vectors** before computing attention.
- For each position p , a **rotation** θ is applied:

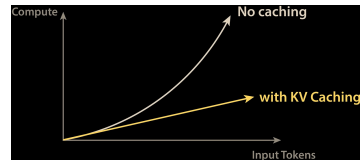
$$q'_p = \theta q_p, \quad k'_p = \theta k_p$$

- **Attention** is then computed as:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{Q'K'^T}{\sqrt{d_k}} \right) V$$

KV cache

- During autoregressive generation, the model produces one token at a time.
- Running the whole decoder block for every token is very expensive.
- **KV cache** stores **Keys** & **Values** from previous transformer computations.
- Speeds up inference by avoiding recomputation of previous tokens.
- Trades **increased memory usage for reduced computation**.



Source: Welch Labs, "How DeepSeek Rewrote the Transformer [MLA]"

Inference Phases in LLMs

■ **LLM inference (generation using a trained model)** is divided into two main stages:

■ **1. Prefill Phase**

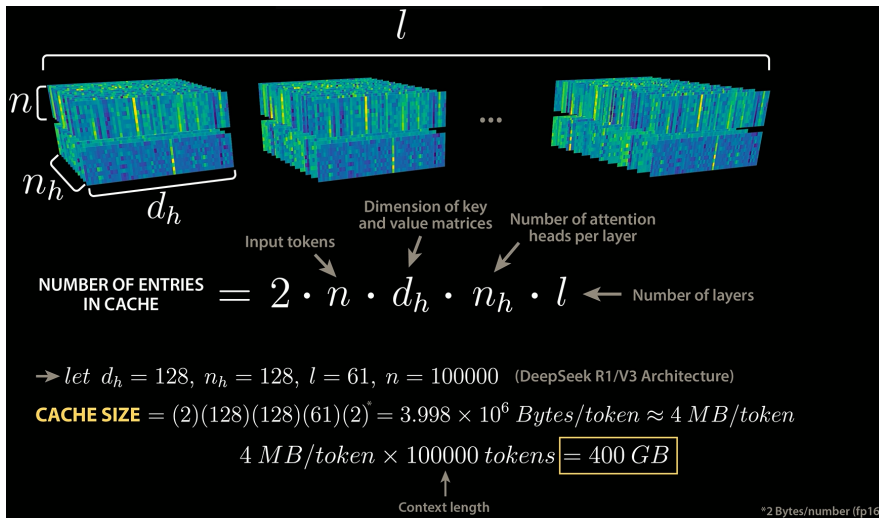
- ▶ Processes the entire input prompt at once
- ▶ Computes hidden states and builds KV cache

■ **2. Decode Phase**

- ▶ Generates output tokens one-by-one (autoregressive)
- ▶ Uses cached K/V to avoid recomputation

Prefill Phase	Decode Phase
Parallel	Sequential
KV cache creation	KV cache reuse
Compute-heavy $O(n^2)$	Memory-heavy

How Large Can the KV Cache Become?



Source: Welch Labs, "How DeepSeek Rewrote the Transformer [MLA]"

Memory Wall in Modern Deep Learning

- The **memory wall** describes the growing gap between:
 - ▶ fast **compute scaling** (GPUs / TPUs)
 - ▶ slow **memory bandwidth scaling**
- Modern accelerators can perform trillions of FLOPs, but are often **stalled waiting for data from memory**.
- In LLMs, performance is frequently **memory-bound rather than compute-bound**:
- Especially critical during **autoregressive decoding**, where:
 - ▶ one token is generated at a time
 - ▶ model weights must be repeatedly streamed from memory
- Modern optimizations (**GQA, MQA, PagedAttention, FlashAttention, Quantization**) primarily aim to **reduce memory bottlenecks**.

Grouped Query Attention (GQA)

- Grouped Query Attention (GQA) is a **direct modification of multi-head attention architecture**.

- In standard multi-head attention, each query head has its own key and value projections:

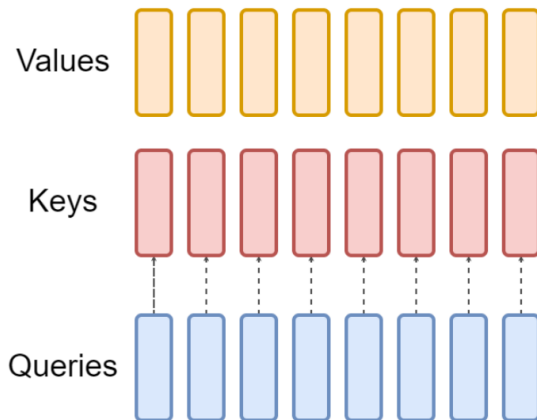
Each head $\rightarrow (Q_i, K_i, V_i)$

- This leads to a large **KV-cache memory footprint**, especially for long sequences.
- In GQA, query heads are divided into groups, and **each group shares the same key and value heads**:

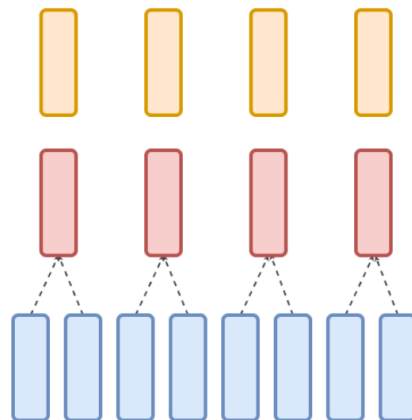
Multiple Q heads $\rightarrow$ shared K, V

Grouped Query Attention (GQA)

Multi-head



Grouped-query



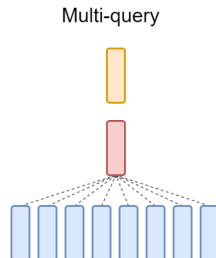
Source: Ainslie et al., "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints" (2023)

Multi-Query Attention (MQA)

- Keeps multiple Query heads, but all heads **share a single Key and Value**:

Multiple Q Heads → One Single Shared KV Head

- This dramatically reduces KV-cache size during decoding.
- MQA trades model expressiveness for significantly lower memory usage.



Source: Ainslie et al., "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints" (2023)

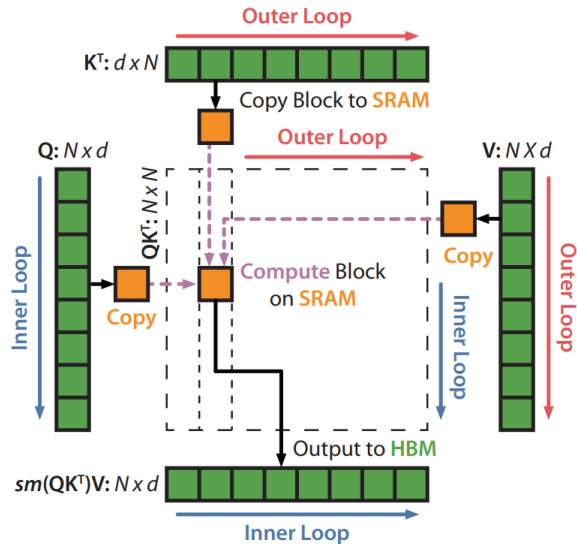
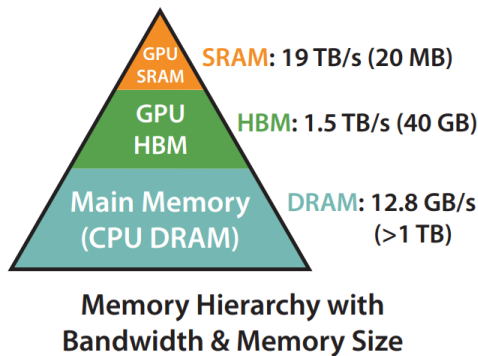
PagedAttention (vLLM)

- PagedAttention is a **KV-cache memory management technique**.
- KV-cache allocation uses **large contiguous memory regions**, causing:
 - ▶ Memory fragmentation
 - ▶ Wasted GPU memory
 - ▶ Lower serving throughput
- PagedAttention treats the KV cache similarly to **virtual memory paging** in operating systems:
 - ▶ KV cache is divided into fixed-size blocks (pages)
 - ▶ Logical token positions are mapped to physical memory pages
 - ▶ Pages can be allocated and reused dynamically
- Benefits:
 - ▶ Higher GPU memory utilization
 - ▶ Reduced fragmentation
 - ▶ Increased throughput for LLM inference

FlashAttention

- Standard attention:
 - ▶ writes QK^T to memory - **Memory Complexity** $O(n^2)$
 - ▶ then reads it again for softmax
- FlashAttention avoids materializing the full attention matrix by computing attention in **small tiled blocks**.
- FlashAttention:
 - ▶ computes attention **block-by-block (tiling)**
 - ▶ keeps intermediate results in fast on-chip memory (SRAM)
 - ▶ avoids storing full $n \times n$ matrix
- It fuses GPU operations and reduces memory reads/writes, making it significantly faster and more memory efficient.

FlashAttention

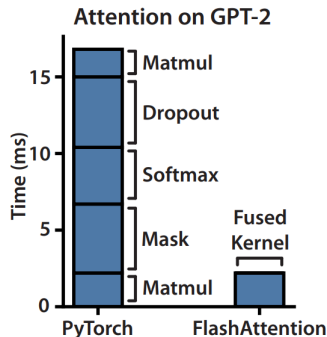


Source: Dao et al., "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness" (2022)

Newest Trends in High-Performance Data Analytics

FlashAttention

- This reduces memory traffic from **quadratic to linear in practice (with respect to memory usage)**.
- Result: **same mathematical output, but much lower memory usage and faster execution.**



Source: Dao et al., "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness" (2022)

Model Quantization

- **Model Quantization reduces the numerical precision** used to store model parameters to lower memory usage and accelerate inference.
- **Memory Required = No of Parameters x Bits per Parameter**
- Instead of storing parameters in **FP32** (32-bit floating point), models use:
 - ▶ FP16 / BF16 (16-bit floating point)
 - ▶ INT8 (8-bit integer)
 - ▶ INT4 (4-bit integer)
- Example for a 70B parameter model:

Precision	Memory
FP16	≈ 140 GB
INT8	≈ 70 GB
INT4	≈ 35 GB

- Benefits: Lower memory bandwidth, Smaller model size, Faster inference

Sparse Attention

- Standard self-attention has **quadratic complexity**: $O(n^2)$ because every token attends to every other token.
- **Sparse attention** reduces computation by restricting each token to attend only to a **subset of tokens**.
- Common sparse patterns:
 - ▶ **Local attention**: tokens attend only to nearby tokens (sliding window)
 - ▶ **Global attention**: selected tokens attend to all tokens
 - ▶ **Random attention**: sparse random connections for long-range links
- Benefit:
 - ▶ Reduces complexity to $O(n)$ or $O(n \log n)$
 - ▶ Enables long-context modeling
- Examples: **Longformer, BigBird**

Linear Attention

- Standard self-attention has **quadratic complexity**: $O(n^2)$ because every token attends to every other token.
- One class of linear attention methods rewrites attention using feature maps:

$$\text{Attention}(Q, K, V) \approx \phi(Q)(\phi(K)^T V)$$

- This removes the need to explicitly compute the $n \times n$ attention matrix.
- Complexity reduces to: $O(n)$
- Trade-off:
 - ▶ Much more memory-efficient
 - ▶ Approximation of softmax attention (not exact)
- Examples: **Performer** (feature-map), **Linformer** (low-rank projection)

Comparison of Modern Approaches

Approach	Improvement over Full Attention	Main Benefit	Trade-off
Full Attention (Baseline)	Baseline, every token attends to every token	Highest expressiveness and accuracy	$O(n^2)$ compute and memory, Large KV cache
Grouped Query Attention (GQA)	Shares KV across groups of query heads	Smaller KV cache, faster decoding	Slightly reduced attention capacity / expressiveness
Multi Query Attention (MQA)	Single shared KV head	Very small KV cache, lowest memory usage, much faster decode	Larger accuracy/quality trade-off than GQA, Reduced expressiveness
PagedAttention	Efficient KV-cache management	Higher throughput and GPU utilization, Less fragmentation	Additional paging / management overhead
FlashAttention	Memory efficient exact attention	Same accuracy, faster execution, lower memory traffic	Requires specialized GPU kernels, Implementation complexity
Quantization	Lower numerical precision	Smaller models, reduced bandwidth, faster inference	Possible accuracy degradation
Sparse Attention	Attend only to selected tokens	Enables much longer contexts, Lower complexity $O(n)$ - $O(n \log n)$	May miss important token interactions
Linear Attention	Approximates softmax attention	Linear $O(n)$ memory and compute scaling	Approximation can reduce accuracy

Attention Interpretability



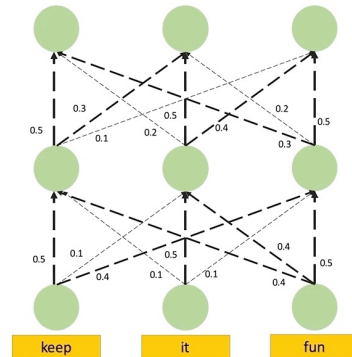
A little girl sitting on a bed with a teddy bear.



A stop sign is on a road with a mountain in the background.

Attention as Graph Edges

- We can model the information flow in the network with a **directed acyclic graph**.
- Input tokens and hidden embeddings are the **nodes**.
- **Edges** are the attentions from the nodes in each layer to those in the previous layer.
- The **weights of the edges** are the attention weights.

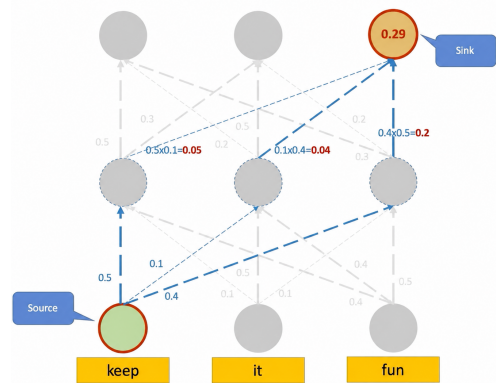
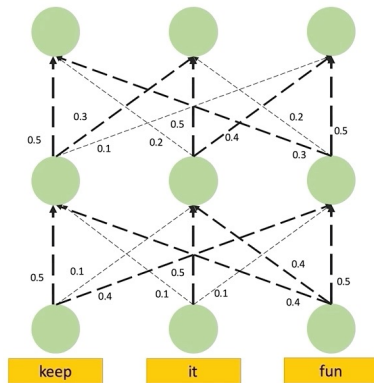


Source: Abnar & Zuidema, "Quantifying Attention Flow in Transformers" (2020)

Attention Roll-Out

- A path from node v at position k in layer l_i , to node u at position m in layer l_j , is a **series of edges** that connect these two nodes.
- Computes how much of the information at v is propagated to u through a particular path by **multiplying the weights of all edges** in that path.
- To compute the total amount of information propagated from v to u , we **sum over all possible paths** between these two nodes.

Attention Roll-Out Example

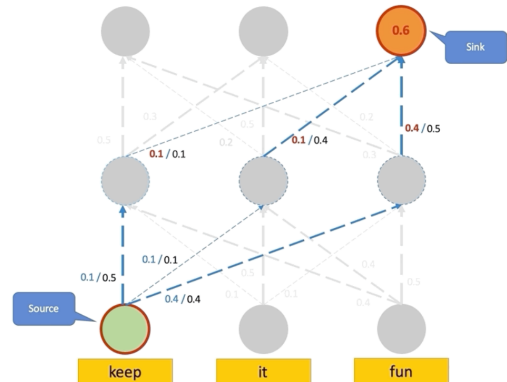
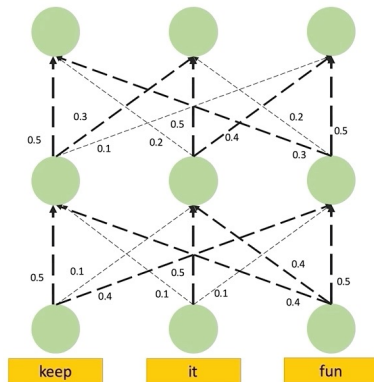


Source: Abnar & Zuidema, "Quantifying Attention Flow in Transformers" (2020)

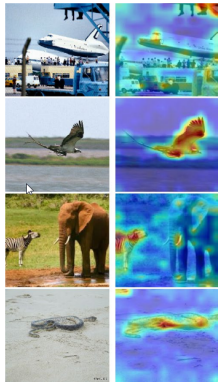
Attention Flow

- Each edge has a **capacity**.
- The flow for each edge should not exceeds its capacity.
- In attention flow, the weight of a single path is **the minimum value of the weights** of the edges in the path.
- Calculating the maximum flow can be tricky due to overlapping paths.

Attention Flow Example

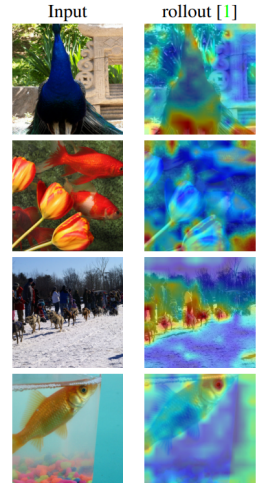
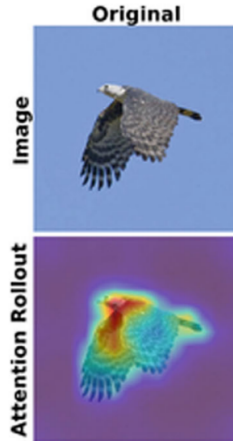


Source: Abnar & Zuidema, "Quantifying Attention Flow in Transformers" (2020)



Input

Rollout
(Abnar and Zuidema, 2020)



Source: Xu et al., "Attribution Rollout: A New Way to Interpret Visual Transformer" (2024)

Conclusion

- Attention solved key limitations of RNNs and LSTMs by enabling direct modeling of **long-range dependencies**.
- The **Transformer architecture** made attention the central mechanism for modern NLP and Large Language Models.
- As context lengths and model sizes grow, memory becomes the **primary bottleneck** during inference.
- Modern architectures increasingly focus on making attention more **dynamic, adaptive, and efficient**.
- As models continue to scale, the future of Transformers depends on smarter allocation of **attention, memory, and compute**.

Final Takeaway

Attention is not just a mechanism, it is the foundation of modern intelligence systems.

References

- Abnar, Samira and Willem H. Zuidema. “Quantifying Attention Flow in Transformers”. In: *arXiv preprint arXiv:2005.00928* (2020). DOI: 10.48550/arXiv.2005.00928. arXiv: 2005.00928 [cs.CL]. URL: <https://arxiv.org/abs/2005.00928>.
- Ainslie, Joshua et al. “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints”. In: *arXiv preprint arXiv:2305.13245* (2023). DOI: 10.48550/arXiv.2305.13245. arXiv: 2305.13245 [cs.CL]. URL: <https://arxiv.org/abs/2305.13245>.
- Beinborn, Lisa and Jonas Mayer Martins. *Language as Data, Interpretability and Bias of Machine Learning Models*. Institute of Computer Science, University of Göttingen.
- Chefer, Hila, Shir Gur, and Lior Wolf. “Transformer Interpretability Beyond Attention Visualization”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2021, pp. 782–791. DOI: 10.1109/CVPR46437.2021.00084. arXiv: 2012.09838 [cs.CV]. URL: <https://arxiv.org/abs/2012.09838>.
- Dao, Tri et al. “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness”. In: *Advances in Neural Information Processing Systems* 35 (2022), pp. 16344–16359. DOI: 10.48550/arXiv.2205.14135. arXiv: 2205.14135 [cs.LG]. URL: <https://arxiv.org/abs/2205.14135>.
- Doshi, Ketan. *Transformers Explained Visually (Part 1): Overview of Functionality*. Towards Data Science. Dec. 2020. URL: <https://towardsdatascience.com/transformers-explained-visually-part-1-overview-of-functionality-95a6dd460452> (visited on 06/09/2026).

References

- Ecker, Alexander. *Machine Learning - Self Attention & Transformers*. Institute of Computer Science, University of Göttingen.
- Kwon, Woosuk et al. "Efficient Memory Management for Large Language Model Serving with PagedAttention". In: *arXiv preprint arXiv:2309.06180* (2023). DOI: 10.48550/arXiv.2309.06180. arXiv: 2309.06180 [cs.LG]. URL: <https://arxiv.org/abs/2309.06180>.
- Lewis, Patrick et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *arXiv preprint arXiv:2005.11401* (2020). DOI: 10.48550/arXiv.2005.11401. arXiv: 2005.11401 [cs.CL]. URL: <https://arxiv.org/abs/2005.11401>.
- Mu, Siyuan and Sen Lin. "A Comprehensive Survey of Mixture-of-Experts: Algorithms, Theory, and Applications". In: *arXiv preprint arXiv:2503.07137* (2025). DOI: 10.48550/arXiv.2503.07137. arXiv: 2503.07137 [cs.LG]. URL: <https://arxiv.org/abs/2503.07137>.
- Olah, Christopher. *Understanding LSTM Networks*. Blog post. 2015. URL: <https://colah.github.io/posts/2015-08-Understanding-LSTMs/> (visited on 06/10/2026).
- Su, Jianlin et al. "RoFormer: Enhanced Transformer with Rotary Position Embedding". In: *arXiv preprint arXiv:2104.09864* (2021). DOI: 10.48550/arXiv.2104.09864. arXiv: 2104.09864 [cs.CL]. URL: <https://arxiv.org/abs/2104.09864>.
- University of Washington. *CSE 543: Deep Learning (Autumn 2023)*. Instructor: Simon S. Du. 2023. URL: <https://courses.cs.washington.edu/courses/cse543/23au/> (visited on 06/09/2026).

References

- Vaswani, Ashish et al. "Attention Is All You Need". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30. Curran Associates, Inc., 2017. arXiv: 1706.03762 [cs.CL]. URL: <https://arxiv.org/abs/1706.03762>.
- Welch Labs. *How DeepSeek Rewrote the Transformer [MLA]*. YouTube video. Mar. 2025. URL: https://www.youtube.com/watch?v=0VLAoVGf_74 (visited on 06/09/2026).
- Xu, Kelvin et al. "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention". In: *arXiv preprint arXiv:1502.03044* (2015). DOI: 10.48550/arXiv.1502.03044. arXiv: 1502.03044 [cs.CV]. URL: <https://arxiv.org/abs/1502.03044>.
- Xu, Li et al. "Attribution Rollout: A New Way to Interpret Visual Transformer". In: *Machine Vision and Applications* (2024). DOI: 10.1007/s00138-024-01524-1. URL: <https://link.springer.com/article/10.1007/s00138-024-01524-1>.